

Gitlab1: DataLab 实验报告

25303090027 朱洪儒

一、终端截图

1. ./check_ops.py bits.c 执行结果

```
Kivacudo@LAPTOP-47LT14J0:/mnt/c/Users/zhu/Desktop/test-git/DataLab-Zhuhongru$ python3 check_ops.py bits.c
signMask: 1/2 operators
bitXor: 7/8 operators
negativePart: 4/6 operators
copyByteWithin: 9/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 17/18 operators
secondLowestZeroBit: 8/8 operators
oddParity: 12/56 operators
rotateRightBits: 12/16 operators
roundEvenPow2: 16/24 operators
midpointTowardFirst: 24/32 operators
isBetweenEitherOrder: 36/48 operators
mul5Sat: 27/30 operators
classifyAdd3: 33/52 operators
floatScaleThreeHalves: 34/60 operators
floatRoundEven: 26/65 operators
float_i2f: 18/40 operators
bitCount: 26/40 operators
bitReverse: 33/34 operators
All 19 functions passed operator checks.
```

2. ./btest 执行结果

```
Kivacudo@LAPTOP-47LT14J0:/mnt/c/Users/zhu/Desktop/test-git/DataLab-Zhuhongru$ ./btest
Score  Rating  Errors  Function
1      1      0      signMask
2      2      0      bitXor
3      3      0      negativePart
4      4      0      copyByteWithin
4      4      0      logicalShift
4      4      0      swapNibblePairs
4      4      0      secondLowestZeroBit
5      5      0      oddParity
5      5      0      rotateRightBits
5      5      0      roundEvenPow2
5      5      0      midpointTowardFirst
7      7      0      isBetweenEitherOrder
7      7      0      mul5Sat
7      7      0      classifyAdd3
7      7      0      floatScaleThreeHalves
10     10     0      floatRoundEven
10     10     0      float_i2f
10     10     0      bitCount
10     10     0      bitReverse
Total points: 110/110
```

二、各函数实现思路

P1: signMask

目标：返回最高位为 1、其余位为 0 的掩码（0x80000000）。

思路：直接将 1 左移 31 位即可得到 0x80000000。利用 `1 << 31` 一步完成，满足最多 2 个操作符的限制。

P2: bitXor

目标：仅使用~和&实现异或。

思路：利用异或的等价形式 $x \oplus y = \sim(x \& y) \& \sim(\sim x \& \sim y)$ 。当 x 和 y 不同时， $\sim(x \& y)$ 为 1， $\sim(\sim x \& \sim y)$ 也为 1；相同时至少有一个为 0。用德摩根律展开即可。

P3: negativePart

目标：若 $x < 0$ 返回 $-x$ ，否则返回 0。

思路： $x \gg 31$ 算术右移得到符号掩码（负数全 1，非负全 0）。 $\sim x + 1$ 即 $-x$ 。用掩码与 $-x$ 相与，非负时结果为 0，负数时得到相反数。

P4: copyByteWithin

目标：将 x 的第 src 个字节复制到第 dst 个字节，其余字节不变。

思路：

计算源字节位置 $srcShift = src \ll 3$ ，目标位置 $dstShift = dst \ll 3$ 。

取出源字节 $byteVal = (x \gg srcShift) \& 0xFF$ 。

构造目标字节的清零掩码 $mask = \sim(0xFF \ll dstShift)$ 。

返回 $(x \& mask) | (byteVal \ll dstShift)$ 。

P5: logicalShift

目标：实现逻辑右移（高位补 0）。

思路：先做算术右移 $x \gg n$ ，然后用掩码清除高 n 位。掩码构造： $(1 \ll 31) \gg n \ll 1$ 得到高 $n+1$ 位为 1 的模式，取反后低 $32-n$ 位为 1。与结果相与即可。

P6: swapNibblePairs

目标：交换每个字节内的高低 4 位。

思路：构造 $mask0F$ （每个字节低 4 位为 1）和 $maskF0$ （每个字节高 4 位为 1）。

将 x 右移 4 位后与 $mask0F$ 相与得到原高 4 位下移的结果；将 x 左移 4 位后与 $maskF0$ 相与得到原低 4 位上移的结果。两者异或（此处位不重叠，等价于或）即为结果。

P7: secondLowestZeroBit

目标：返回第二个最低的 0 位对应的掩码。

思路：对 x 取反得到 $a = \sim x$ ，其中 1 表示原 x 的 0 位。 $low1 = a \& (\sim a + 1)$ 取最低的 1 位（即最低的 0 位）。 $a2 = a \oplus low1$ 清除该位。再对 $a2$ 取最低位 $low2 = a2 \& (\sim a2 + 1)$ 即第二个 0 位。若不存在则返回 0。

P8: oddParity

目标：当 x 中 1 的个数为偶数时返回 1，否则返回 0。

思路：采用折叠异或法。依次 $x \oplus = x \gg 16$ 、 $\gg 8$ 、 $\gg 4$ 、 $\gg 2$ 、 $\gg 1$ ，最终 x 的最低位即为所有位的异或和（奇偶校验位）。若 1 的个数为偶数，异或和为 0，返回 $!(x \& 1) = 1$ ；否则返回 0。

P9: tateRightBits

目标：将 x 循环右移 n 位。

思路：循环右移 = 逻辑右移 n 位（低部分）| 左移 $(32-n)$ 位（高部分）。低部分用逻辑右移掩码清除高位；高部分左移位数用 $(32 + \sim n + 1) \& 31$ 计算，即 $(32 - n) \bmod 32$ ，保证 $n=0$ 时不移位。

P10: roundEvenPow2

目标：将非负 x 舍入到最接近的 2^n 的倍数，中间值时取商为偶数的那个。

思路：

$\text{pow2} = 1 \ll n$, $\text{scissorleft} = \text{pow2}-1$ （低位掩码）。

$\text{sthleft} = x \& \text{scissorleft}$ 为余数部分。

判断是否进位：若余数 $> \text{pow2}/2$ 则进位；若余数 $== \text{pow2}/2$ 则看商的最低位（偶数不进位，奇数进位）。用 newleft 计算进位后的增量。

最终结果 = 进位增量 + $(x \& \sim(\text{pow2}-1))$ （保留高位部分）。

P11: midpointTowardFirst

目标：返回 $(x+y)/2$ 的精确中点，无溢出；中间值时取靠近 x 的整数。

思路：

用 $(x \& y) + ((x \oplus y) \gg 1)$ 计算向下取整的中点（避免溢出）。

判断 x 是否大于等于 y ：比较符号位和差值符号。

判断 $x+y$ 是否为奇数： $(x \oplus y) \& 1$ 。

若 $x \geq y$ 且为奇数，向上取整 $(+1)$ ，使结果更接近 x ；否则向下取整。

P12: isBetweenEitherOrder

目标：判断 x 是否在 a 、 b 构成的闭区间内（端点可任意顺序）。

思路：分别计算四个比较： $x \geq a$ 、 $x \leq b$ 、 $x \geq b$ 、 $x \leq a$ 。使用符号位技巧实现无溢出比较： $\text{diff} = x - a$ ，若 x 与 a 同号且 diff 与 x 异号则发生溢出，需修正。最终 $(A \& B) \mid (C \& D)$ 即为结果。

P13: mul5Sat

目标：计算 $x*5$ ，溢出时饱和到 INT_MAX 或 INT_MIN 。

思路：

$x4 = x \ll 2$, $\text{sum} = x + x4$ 。

检测左移溢出： $(x4 \gg 2) \wedge x$ 非零则溢出。

检测加法溢出： x 与 $x4$ 同号且 sum 与 x 异号。

溢出时根据 x 的符号返回 INT_MAX 或 INT_MIN ；否则返回 sum 。

P14: classifyAdd3

目标：判断 $x+y+z$ 的精确和是 $> \text{INT_MAX}$ （返回 1）、 $< \text{INT_MIN}$ （返回 -1）还是正常（返回 0）。

思路：分两步加法，每步检测溢出方向和符号。

$s1 = x + y$ ，检测溢出 ov1 及其方向（正溢出/负溢出）。

$s2 = s1 + z$ ，检测溢出 ov2 及其方向。

最终分类：正溢出次数减负溢出次数，得到 1、-1 或 0。

P15: floatScaleThreeHalves

目标：计算浮点数 $f \times 3/2$ ，使用 round-to-nearest-even。

思路：

提取符号、指数、尾数。

NaN/Inf 直接返回。

非规格化数：尾数 $m \times 3$ ，然后除以 2 并 round-to-even。若进位到 2^{23} 则转为规格化数。

规格化数：尾数（含隐含 1） $\times 3$ ，除以 2 并 round-to-even。若超过 2^{24} 则指数 +1 并再次舍入。溢出到 Inf 时返回 $\pm \text{Inf}$ 。

P16: floatRoundEven

目标：将浮点数舍入到最接近的整数（中间值取偶数），返回浮点表示。

思路：

提取符号、指数、尾数。NaN/Inf 直接返回。

非规格化数绝对值 < 1 ，直接舍入到 ± 0 。

实际指数 $e = \text{exp} - 127$ ：

$e \geq 23$ ：本身就是整数，直接返回。

$e < -1$ ：绝对值 < 0.5 ，舍入到 ± 0 。

$e == -1$ ： $0.5 \leq |x| < 1$ ，单独处理，根据 sticky 和 lsb 决定舍入到 0 还是 1。

$0 \leq e \leq 22$ ：右移尾数舍入到整数，再转回浮点表示。

P17: float_i2f

目标：将 int 转换为 float 的位表示。

思路：

$x == 0$ 返回 0；提取符号并取绝对值。

左移绝对值直到最高位为 1，同时调整指数。

舍掉低 8 位，使用 round-to-even：guard 位、sticky 位、lsb 位决定是否进位。

若进位到第 24 位，指数 +1 并右移一位。

组合符号、指数、尾数返回。

P18: bitCount

目标：统计 x 中 1 的个数。

思路：采用分治并行计数法：

每 2 位一组，统计每组 1 的个数 ($x = (x \& 0x55) + ((x \gg 1) \& 0x55)$)。

每 4 位一组，合并相邻 2 位组 ($\& 0x33$)。

每 8 位一组，合并 ($\& 0x0F$)。

每 16 位一组，合并 ($\& 0xFF$)。

最后高低 16 位相加，取低 6 位即为结果。

P19: bitReverse

目标：反转 32 位整数的所有位。

思路：采用分治交换法：

相邻 1 位交换 (m55)。

相邻 2 位交换 (m33)。

相邻 4 位交换 (m0F)。

相邻 8 位交换 (mFF)。

相邻 16 位交换 (m16)。

每一步用掩码取出两部分，分别移位后合并，最终得到完全反转的结果。